

Teoria dos Números: GCD, LCM e Aritmética Modular

Universidade Estadual do Sudoeste da Bahia - UESB

Aula 2 — 2026.1

Maior Divisor Comum (GCD / MDC)

O **Greatest Common Divisor (GCD)** de dois inteiros a e b é o maior inteiro que divide ambos sem deixar resto. Para computá-lo eficientemente, utilizamos o **Algoritmo de Euclides**:

$$\text{gcd}(a, b) = \begin{cases} a & \text{se } b = 0 \\ \text{gcd}(b, a \bmod b) & \text{se } b \neq 0 \end{cases}$$

- ▶ **Complexidade de Tempo:** $\mathcal{O}(\log(\min(a, b)))$, extremamente rápido!
- ▶ **Associatividade:** $\text{gcd}(a, \text{gcd}(b, c)) = \text{gcd}(\text{gcd}(a, b), c)$. Funciona para N números.

Implementação vs Built-in

Você pode codificar a recursão manualmente ou usar as ferramentas nativas da linguagem:

```
// Implementacao manual classica (C++ de forma recursiva)  
int gcd(int a, int b) {  
    return b == 0 ? a : gcd(b, a % b);  
}
```

Facilidades das versões modernas:

- ▶ **C++14:** Função interna do compilador: `__gcd(a, b)`.
- ▶ **C++17:** Função oficial da biblioteca padrão na biblioteca `<numeric>`:
`std::gcd(a, b)`.

Menor Múltiplo Comum (LCM / MMC)

O **Least Common Multiple (LCM)** de dois inteiros a e b é o menor inteiro positivo que é divisível por ambos. Podemos calculá-lo diretamente a partir do GCD usando a seguinte propriedade matemática:

$$\text{lcm}(a, b) = \frac{a \cdot b}{\text{gcd}(a, b)}$$

Atenção à Ordem das Operações (Overflows):

- ▶ Em código, faça a divisão **antes** da multiplicação para evitar que $a \cdot b$ estoure o limite das variáveis (*overflow*):
- ▶ `int lcm = (a / std::gcd(a, b)) * b;`
- ▶ **C++17**: Assim como o GCD, existe o `std::lcm(a, b)` em `<numeric>`.
- ▶ O LCM também é **associativo**.

Vamos praticar

// Abra o seu editor e vamos codar

Consulte a lista de exercícios recomendados.

Aritmética Modular

Em problemas, os resultados podem ser gigantescos. Em vez de operar com inteiros puros, trabalhamos com seus **restos** ao dividir por um módulo m (Operação *modulo* m).

Exemplo: Se $m = 23$, em vez de usarmos $x = 247$, usamos $247 \bmod 23 = 17$.

- ▶ **Módulos Comuns:** Problemas costumam exigir a resposta sob um primo grande para evitar *overflow* e permitir o inverso modular. Os mais famosos são:
 - ▶ $10^9 + 7$
 - ▶ 998 244 353

Propriedades da Aritmética Modular

O operador modular pode ser distribuído ao longo das operações aritméticas básicas. Isso nos permite aplicar o módulo a cada passo do cálculo:

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$$

$$a^b \bmod m = (a \bmod m)^b \bmod m$$

Dica de Competição: No caso da subtração, somamos $+m$ antes do módulo final para garantir que o resultado não seja um número negativo no C++.

Accepted

Tempo: 0.01s — Memória: 120 KB

Bons estudos e nos vemos na Aula 3.